Collections

April 14, 2026

1 Collections

1.1 Contents

1.2 Notes

1. The notebooks are largely self-contained, i.e, if a symbol appears, an explanation will be present at some point in the notebook.
    - Most often there will be links to the cell where the symbols are explained
    - If the symbols are not explained in this notebook, a reference to the appropriate notebook will be provided
2. **Github does a poor job of rendering this notebook**. The online render of this notebook is missing links, symbols, and notations are badly formatted. It is advised to clone a local copy (or download the notebook) and open it locally.
3. The Numbers notebook may be referred after this notebook for more information on real number, integer, complex number notations.

1.3 Importing Libraries

[2]:
```
import itertools
import math
```

Sets

Introduction

A set is an unordererd collection of objects in which repetition is forbidden. Order does NOT matter. Denoted by

$$\{a, b, c\}$$

Notation: {2,3,7} is a set of three integers

Large sets can be specified using a ellipses to show a continuing pattern: {2,4,6,8, ...}

[3]:
```
{12,4,21,21}
```

[3]:
```
{4, 12, 21}
```

Some literature introduce concepts using partially ordered sets or Posets. A partial order is any binary relation that is reflexive (each element is comparable to itself), antisymmetric (no two different elements precede each other), and transitive (the start of a chain of precedence relations must precede the end of the chain). A partially oredered set P will be denoted along with its binary relation such as: $(P, \leq)$

Membership

Membership in a set is indicated using $\in$ which is read as 'belongs to' or 'is an element of'. So

$$x \in A$$

can be read as object x is an element of set A.

Note: sometimes the backward $\in$ is used in the same way, such that

$$A \ni x$$

means exactly the same as above. It is sometimes used to represent the phrase 'such that' but its more common to represent 'such that' with **s.t**

[4]:
```
A = {5,6,7}
x = 5

x in A
```

[4]: True

The opposite, i.e non-Membership, is indicated using $\notin$ which is read as 'not an element of' or 'does not belog to'. So

$$x \notin A$$

can be read as object x is NOT an element of set A.

[5]:
```
A = {5,6,7}
x = 4

x not in A
```

[5]: True

Null set

A set containing no elements is called a null set or an empty set. It is most commonly denoted by

$$\emptyset$$

or

$$\emptyset$$

The notation

$$\{\}$$

is also used when the context makes it clear that the object is a set.

[6]:
```
A = set()

A == set()
```

[6]: True

Real set

A set containing real numbers is called a real number set. It is represented by:

$$\mathbb{R}$$

Membership of variable x in the real number set is represented as:

$$x \in \mathbb{R}$$

[7]:
```
x = 3.8

# x  R: check if x is a real number
isinstance(x, (float, int))
```

[7]:
```
True
```

Note 1: This notation can also be used to represent higher dimensional variables $\mathbf{x} \in \mathbb{R}^n$

Please see the **Numbers** notebook for a numbers perspective on this topic and the **Linear Algebra** notebook for vectors prespective.

Integer set

A set containing only integers is called an integer set. It is represented by:

$$\mathbb{Z}$$

Membership of variable x in the integer set is represented as:

$$x \in \mathbb{Z}$$

Common explicit variants are

$$\mathbb{Z}_{\geq 0}, \mathbb{Z}_{>0}, \mathbb{Z}_{\leq 0}, \mathbb{Z}_{<0}$$

for nonnegative, positive, nonpositive, and negative integers respectively. Decorations such as $\mathbb{Z}^+$ and $\mathbb{Z}^-$ are also used, but their meanings vary by author.

[8]:
```
x = 5

# x  Z: check if x is an integer
isinstance(x, int)
```

[8]:
```
True
```

Complex set

A set containing only complex numbers is called a complex number set. It is represented by:

$$\mathbb{C}$$

Membership of variable x in the complex number set is represented as:

$$x \in \mathbb{C}$$

For more information, see the Numbers notebook

[9]:
```
z = complex(3, 5)

print('real part:', z.real, ', imaginary part:', z.imag)
isinstance(z, complex)
```

```
real part: 3.0 , imaginary part: 5.0
```

[9]:
```
True
```

Set builders notation

The set builder notation can be used to represent sets in a more consistent manner. The set builder notation will include a couple of statements within curly braces that generally denotes the elements of the built set and it's conditions. For example

$$A = \{x \in \mathbb{Z} : 1 \leq x \leq 100\}$$

This can be read as a set of integers (See: Integer set) where each element is represented using a dummy variable x (i.e all elements of set A are integers). The colon then describes additional conditions for x, which in this case is that x has to be between 1 and 100 **inclusive**.

Sometimes, instead of a colon, a pipe can be used . **This DOES not mean conditioning as used in probability notations**. The meaning is the same as above, the colon and pipe are interchangeable from a notation perspective. The above notation is equivalent to:

$$\{x \in \mathbb{Z} \mid 1 \leq x \leq 100\}$$

Sometimes the conditions can be described in simple words. It has the same meaning as the above as well:

$$\{x \in \mathbb{Z} \mid \text{x is greater than or equal to 1 but less than or equal to 100}\}$$

[10]:
```
# {x   Z : 1   x   100}
A = {x for x in range(-200, 201) if 1 <= x <= 100}

print('A:', sorted(A)[:5], '...', sorted(A)[-5:])
print('|A|:', len(A))
```

```
A: [1, 2, 3, 4, 5] … [96, 97, 98, 99, 100]
|A|: 100
```

When the context is clear, the preamble may be skipped. Alternatively, the preamble may be used to just introduce the dummy variable such that:

$$\{x : 1 \leq x \leq 100\}$$

or

$$\{x \mid 1 \leq x \leq 100\}$$

The conditions can be sophisticated and complex at times and can require careful reading to understand the nuances. For example: some of the conditions may actually be expressions that need to be evaluated such as:

$$C = \{(x, y) \mid x \in A, y \in B, x + y = 6\}$$

Breaking this down, this set builder notation builds a set of ordered pairs of x and y such that x takes values from set A and y takes values from set B and $x + y$ SHOULD sum to 6, . It should be noted that $(x, y) \neq (y, x)$ since the paranthesis suggests that (x, y) is a list. For more on lists, see: introduction to Lists

Although this set builder notation is readable it can also be written using logic notations, using Logical And $\wedge$:

$$C = \{(x, y) \mid x \in A \wedge y \in B \wedge x + y = 6\}$$

For more information on Logical And, see the Logic notebook

[11]:
```
A = {-2, 4, 7}
B = {2, 8, 9}

# {(x, y) : x   A, y   B, x + y = 6}
C = set()
for x in A:
    for y in B:
        if x + y == 6:
            C.add((x, y))

print('A:', A, ', B:', B)
print('C:', C)
```

```
A: {4, -2, 7} , B: {8, 9, 2}
C: {(-2, 8), (4, 2)}
```

Set equality

Two sets A and B are equal if they have the same elements.

$$A = B$$

[12]:
```
A = {3,5,6,7}
B = {3,5,6,7}

A == B
```

[12]: True

Subset

Consider two sets A and B. A is considered a subset of B if all elements of A are also elements in B. A may also be equal to B.

Denoted by:

$$A \subseteq B$$

Note: Strict subsets can be denoted by $\subset$. This denotes only subset with no equality of the subsets. Consider sets A and B where A is a subset of B but never equal to B:

$$A \subset B$$

However, this isn't universally accepted and some authors use $\subset$ and $\subseteq$ interchangeably

[13]:
```
A = {3,5,6,7}
B = {3,5,6,7,8,9,10}
C = {3,5,6,7}

A.issubset(B), A.issubset(C), A == B, A == C
```

[13]: (True, True, False, True)

Note: $A \subseteq B$ is not the same as $A \in B$. The subset relation concerns **sets** A and B, whereas membership $x \in B$ concerns **elements** in sets.

[14]:
```
A = {4}
B = {4}

print('A is a subset of B: ',A.issubset(B))

print('\nA belongs to B', A in B)

for x in A:
    print('\nElement in A belongs to B: ', x in B)
```

```
A is a subset of B:  True

A belongs to B False

Element in A belongs to B:  True
```

Superset

Consider two sets A and B. A is considered a superset of B if all elements of B are also elements in A. A may also be equal to B.

Denoted by:

$$A \supseteq B$$

Note: Strict supersets can be denoted by $\supset$. This denotes only superset with no equality of the subsets. Consider sets A and B where A is a superset of B but never equal to B:

$$A \supset B$$

However, this isn't universally accepted and some authors use $\supset$ and $\supseteq$ interchangeably

[15]:
```
A = {3,5,6,7,8,9,10}
B = {3,5,6,7}
C = {3,5,6,7,8,9,10}

A.issuperset(B), A.issuperset(C), A == B, A == C
```

[15]:
```
(True, True, False, True)
```

Union

Consider two sets A and B. The union of A and B is the set of all elements that belong to A, to B, or to both.

Denoted by:

$$A \cup B$$

$$A \cup B = \{x : x \in A \text{ or } x \in B\}$$

[16]:
```
A = {3,5,6,7,8,9,10}
B = {3,5,6,         11,12,13,14,15}

A.union(B)
```

[16]:
```
{3, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15}
```

Disjoint Union

Consider sets A and B. A disjoint union indicates that the sets being combined do not share elements.

This may be denoted by symbols such as $\uplus$, $\sqcup$, or sometimes $\oplus$, depending on the author and the subject.

For example,

$$A_1 \uplus A_2 \uplus \cdots \uplus A_n = B$$

means:

- $$A_i \cap A_j = \emptyset \quad \forall i \neq j$$
- $$A_1 \cup A_2 \cup \cdots \cup A_n = B$$

For more information on $\forall$, see the Logic notebook.

[17]:
```
def disjoint_union(A, B):
    if A.intersection(B) == set():
        return A.union(B)
    else:
        return 'Invalid'

A = {7, 8, 9, 10}
B = {3, 5, 6, 11, 12, 13, 14, 15}

disjoint_union(A, B)
```

[17]:
```
{3, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15}
```

Intersection

Consider two sets A and B. The intersection of A and B are the set of elements that they have in common.

Denoted by:

$$A \cap B$$

[18]:
```
A = {3,5,6,7,8,9,10}
B = {3,5,6,        11,12,13,14,15}

A.intersection(B)
```

[18]:
```
{3, 5, 6}
```

Difference or relative complement

Consider two sets A and B. The difference of A and B is a set of all elements in A that are not in B.

Denoted by:

$$A - B$$

Since this can be ambiguous and may **erroneously** be interpreted as $a - b$ where $a \in A$ and $b \in B$, it can also be denoted as a relative complement:

$$A \setminus B$$

[19]:
```
A = {3,5,6,7,8,9,10}
B = {3,5,6,        11,12,13,14,15}

A - B
```

[19]:
```
{7, 8, 9, 10}
```

Complement

The absolute complement of a set A is all elements that are not in A. This can mean a very large space of elements and is denoted various ways:

$$A^c$$

or

$$\overline{A}$$

or

$$A'$$

Symmetric Difference

Consider two sets A and B. The symmetric difference of A and B is the set of elements that belong to A or B, but not to both.

Denoted by:

$$A \Delta B$$

$$A \Delta B = (A - B) \cup (B - A)$$

[20]:
```
A = {3,5,6,7,8,9,10}
B = {3,5,6,        11,12,13,14,15}

A.symmetric_difference(B)
```

[20]:
```
{7, 8, 9, 10, 11, 12, 13, 14, 15}
```

Cardinality

Consider set A, the cardinality is the number of elements in A. Denoted by:

$$| A |$$

[21]:
```
A = {3, 5, 6, 7, 8, 9, 10}

# |A|
```

```
len(A)
```

[21]: 7

Cartesian Product

Consider two sets A and B, the cartesian product is the set of all oredered pairs of elements in A and B:

$$A \times B = \{(a, b) \mid a \in A, b \in B\}$$

Here (a,b) is a list (ordered pair) so $(a, b) \neq (b, a)$. See Lists: Introduction

[22]:
```
A = {3,5,6}
B = {7,8,9}

set(itertools.product(A,B))
```

[22]: {(3, 7), (3, 8), (3, 9), (5, 7), (5, 8), (5, 9), (6, 7), (6, 8), (6, 9)}

As an extension of the cartesian product, the notation A^2 is cartesian product of A with A, so it is:

$$A^2 = \{(x, y) \mid x, y \in A\}$$

[23]:
```
A = {3,5,6}

set(itertools.product(A,repeat=2))
```

[23]: {(3, 3), (3, 5), (3, 6), (5, 3), (5, 5), (5, 6), (6, 3), (6, 5), (6, 6)}

A further extension of the cartesian product, the notation A^n is the set of all n-tuples of elements of A:

$$n \in \mathbb{Z}_{>0},\ A^n = \{(a_1, a_2, ..., a_n) \mid a_1, a_2, ..., a_n \in A\}$$

(See: Integer set)

[24]:
```
A = {3,5}
n = 3

set(itertools.product(A,repeat=n))
```

[24]:
```
{(3, 3, 3),
 (3, 3, 5),
 (3, 5, 3),
 (3, 5, 5),
 (5, 3, 3),
 (5, 3, 5),
 (5, 5, 3),
 (5, 5, 5)}
```

Power Set

Consider a set A, the power set of A is the set of all subsets of A. Denoted by the Weierstrass symbol:

$$\wp(A)$$

or

$$2^A$$

[25]:
```
A = {2, 3, 4}

# P(A): all subsets of A
P = []
for size in range(len(A) + 1):
    for s in itertools.combinations(A, size):
        P.append(set(s))

print('A:', A)
print('P(A):', P)
print('|P(A)| = 2^|A| =', 2**len(A))
```

```
A: {2, 3, 4}
P(A): [set(), {2}, {3}, {4}, {2, 3}, {2, 4}, {3, 4}, {2, 3, 4}]
|P(A)| = 2^|A| = 8
```

Set Exponentiation

If A and B are sets, then B^A denotes the set of all functions from A to B:

$$B^A = \{f \mid f : A \to B\}$$

For more information on the notation $f : A \to B$, see the Functions notebook

If A and B are finite sets with $|A| = a$ and $|B| = b$, then the number of such functions is:

$$|B^A| = |B|^{|A|} = b^a$$

This is the reason for the exponential notation: the cardinality (See: Cardinality) of B^A equals b raised to the power a.

[ ]:
```
A = {1, 2}
B = {'x', 'y', 'z'}

# All functions from A to B: each element of A maps to an element of B
functions = []
for values in itertools.product(B, repeat=len(A)):
    f = dict(zip(A, values))
```

```
    functions.append(f)


print('A:', A, ', |A|:', len(A))
print('B:', B, ', |B|:', len(B))
print('|B^A| = |B|^|A| =', len(B), '^', len(A), '=', len(B)**len(A))
print('Number of functions from A to B:', len(functions))
print(functions)
```

```
A: {1, 2} , |A|: 2
B: {'x', 'z', 'y'} , |B|: 3
|B^A| = |B|^|A| = 3 ^ 2 = 9
Number of functions from A to B: 9
[{1: 'x', 2: 'x'}, {1: 'x', 2: 'z'}, {1: 'x', 2: 'y'}, {1: 'z', 2: 'x'}, {1:
'z', 2: 'z'}, {1: 'z', 2: 'y'}, {1: 'y', 2: 'x'}, {1: 'y', 2: 'z'}, {1: 'y', 2:
'y'}]
```

Infimum, Supremum, Max, Min

If A is a set of real numbers, then the largest and smallest numbers are shown as:

$$\max A, \min A$$

However, it is not always possible to evaluate the largest and/or smallest of elements of a set. For example: let $B = \{x \in \mathbb{R} \mid -1 < x < 1\}$

It can be seen $max B,min B $ **cannot** be evaluated since -1 and 1 do not belong to the set B. In such cases, the bounds of B are evaluated using the concept of Supremum (also called *least upper bound*) and Infimum (also called *greatest lower bound*). These are denoted as

$$\sup B = 1, \inf B = -1$$

The supremum and infimum are properties **OF** the set B and may not neccesarily be **IN** the set B. It may also be denoted using:

$$\text{lub} B = 1, \text{glb } B = -1$$

[27]:
```
A = set({2,8,11})

min(A), max(A)
```

[27]:
```
(2, 11)
```

[28]:
```
# B represents the open interval (-1, 1)
# inf(B) = -1 and sup(B) = 1 but neither belongs to B
B = [-0.999, -0.5, 0, 0.5, 0.999]
```

```
print('min(B):', min(B), ' (approaches inf = -1)')
print('max(B):', max(B), ' (approaches sup = 1)')
```

```
min(B): -0.999  (approaches inf = -1)
max(B): 0.999  (approaches sup = 1)
```

Note 1:

The concepts of supremum and infimum apply in ordered settings, but not every partially ordered set has a least upper bound or greatest lower bound for every subset.

- When the least upper bound and greatest lower bound exist, they are denoted by $\sup A$ and $\inf A$.
- If the maximum and minimum exist, then $\max A = \sup A$ and $\min A = \inf A$.

For example, the extended real line $\mathbb{R} \cup \{-\infty, \infty\}$ has both a minimum $-\infty$ and a maximum ∞.

Another example is

$$A = \{x^{-1} \mid x \in \mathbb{N}\}$$

where $\mathbb{N}$ is the set of natural numbers (**see the Numbers notebook for more details on natural numbers**). In this case:

$$\max A = \sup A = 1, \quad \text{at } x = 1$$

but the minimum does not exist, while

$$\inf A = 0$$

Note 2:

The wedge $\wedge$ and vee $\vee$ symbols are also used in the context of pairwise maximum and minimum of two values:

$$x \vee y = \max\{x, y\}$$
$$x \wedge y = \min\{x, y\}$$

Note: These are the same symbols used for Logical And and Or. Here they take on a different meaning in the context of ordered sets. See the **Logic notebook** for the Boolean interpretation.

[29]:
```
x = 3
y = 7

print('x   y = max{x, y} =', max(x, y))
print('x   y = min{x, y} =', min(x, y))
```

```
x   y = max{x, y} = 7
x   y = min{x, y} = 3
```

Lists or n-tuples

Introduction

Note: This refers to the mathematical definition of a list.

A list is an ordered collection of objects in which repetition is permitted. Lists are denoted using paranthesis:

$$A = (2, 7, 2, 8, 11)$$

Order matters in lists and repetiton is allowed such that $(a_1, a_1, a_2) \neq (a_1, a_2, a_1)$

A list of n elements is sometimes called an $n-$tuple.

[30]:
```
# A = (a2, a1, a1)
A = [2,1,1] # or A = (2,1,1) since python lists and python tuples are␣
 ↪equivalent to
            # the mathematical definition of list

# B = (a1, a2, a2)
B = [1,2,2] # or B = (1,2,2) since python lists and python tuples are␣
 ↪equivalent to
            # the mathematical definition of list

A == B
```

[30]:
```
False
```

[31]:
```
#Just for comparison, sets are unordered collections
set(A) == set(B)
```

[31]:
```
True
```

When listing 2-tuples using paranthesis, the notation for it: (a, b) may look like an open interval (**See Numbers notebook for more info on intervals**).

To ensure better representation for $n-$tuples and clear distinction between open intervals, some authors also denote it using angle brackets:

$$\langle a, b \rangle$$

This is not limited to 2-tuples, it can be used for n-tuples: $\langle a, b, c, d, e \rangle$ which is equivalent to (a, b, c, d, e) shown above.

Set of lists

A set consisting of ordered pairs is quite commonly used in math literature. This is denoted using set builder notation with an $n-$tuple in the preamble.

$$C = \{(a, b) \mid a \in A, b \in B\}$$

[32]:
```
A = {2, 3, 4}
B = {5, 6, 7}

# A x B as a set of 2-tuples (ordered pairs)
C = set(itertools.product(A, B))

print('C:', C)
print('|C|:', len(C))
print('example element:', (2, 5), ', type:', type((2, 5)))
```

```
C: {(2, 7), (3, 7), (4, 6), (4, 5), (2, 6), (3, 6), (2, 5), (4, 7), (3, 5)}
|C|: 9
example element: (2, 5) , type: <class 'tuple'>
```

Aggregation symbols

Big Sum

The symbol $\sum$ is used to represent a sum of a collection. The sum notation is typically used as:

$$\sum_{i=\text{start}}^{stop} \text{expression involving i}$$

For example:

$$\sum_{i=1}^{5} x^i$$

which after explicitely expanding the sum will look like:

$$\sum_{i=1}^{5} x^i = x^1 + x^2 + x^3 + x^4 + x^5$$

[33]:
```
# Σ(i=1 to 5) x^i, with x = 3
x = 3
total = 0

for i in range(1, 6):
    total += x ** i
    print(f'i={i}: x^i = {x**i}, running sum = {total}')

total
```

```
i=1: x^i = 3, running sum = 3
i=2: x^i = 9, running sum = 12
i=3: x^i = 27, running sum = 39
i=4: x^i = 81, running sum = 120
i=5: x^i = 243, running sum = 363
```

[33]: 363

If the stop condition is ∞, the summation keeps going on without end, an endless aggregation can be denoted using elipsis as shown. Assume a function F such that:

$$F(m) = \sum_{n=0}^{m} \frac{1}{2^n} = 1 + \frac{1}{2} + \frac{1}{4} + ... + \frac{1}{2^{m-1}} + \frac{1}{2^m}$$

Then

$$F(\infty) = \sum_{n=0}^{\infty} \frac{1}{2^n} = 1 + \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + ...$$

When practically computing this sum, it's obviously impossible to keep summing to infinity, the computation will never end. In such cases, a tolerance level δ is defined as the threshold for the increase in magnitude of the sum per iteration, below which the computation is terminated. This is generally shown in a form of a equation such as:

$$F(i) - F(i-1) \leq \delta$$

In this case

$$F(i) - F(i-1) = \frac{1}{2^i}$$

so the tolerance value for termination will be:

$$\frac{1}{2^i} \leq \delta$$

For more information defining functions F, see the Functions notebook

[34]:
```
# Σ(n=0 to ∞) 1/2^n converges to 2
total = 0

for n in range(20):
    total += 1 / 2 ** n

print(f'partial sum (20 terms): {total}')
print(f'exact limit: 2.0')
```

```
partial sum (20 terms): 1.9999980926513672
exact limit: 2.0
```

Big Product

The symbol $\prod$ is used to represent products of a collection and it is analogous to the big sum. The big product notation is typically used as:

$$\prod_{i=\text{start}}^{\text{stop}} \text{expression involving } i$$

For example:

$$\prod_{i=0}^{5}(2i+1)$$

which after explicitly expanding the product looks like:

$$\prod_{i=0}^{5}(2i+1) = 1 \times 3 \times 5 \times 7 \times 9 \times 11$$

[35]:
```
prod = 1

for i in range(6):
    prod *= (2*i+1)
    print('Iteration: ',i,', value: ',(2*i+1),', product: ',prod)

prod
```

```
Iteration:  0 , value:  1 , product:  1
Iteration:  1 , value:  3 , product:  3
Iteration:  2 , value:  5 , product:  15
Iteration:  3 , value:  7 , product:  105
Iteration:  4 , value:  9 , product:  945
Iteration:  5 , value:  11 , product:  10395
```

[35]:
```
10395
```

Sometimes a summation expression is presented without any indication as to which variable is the dummy variable and without specifying upper or lower limits for the index. In such cases, it is often the case that a repeated variable is the summation index. For example, if the expression

$$\sum \binom{n}{k} x^k$$

is encountered, then it is likely that k is the summation index. Since $\binom{n}{k}$ is defined only for $k \geq 0$ and is zero for $k > n$, it is safe to assume k ranges from 0 to n:

$$\sum \binom{n}{k} x^k = \sum_{k=0}^{n} \binom{n}{k} x^k$$

In general, if no upper or lower bounds are given for the index, the sum is over all allowable values for that index.

[36]:
```
# Implicit bounds: sum of C(n,k) * x^k for k = 0 to n
# This is the binomial theorem: (1 + x)^n

n = 4
x = 2
```

```
result = sum(math.comb(n, k) * x**k for k in range(n + 1))
print('sum of C(n,k) * x^k for n =', n, ', x =', x, ':', result)
print('(1 + x)^n =', (1 + x)**n)
```

```
sum of C(n,k) * x^k for n = 4 , x = 2 : 81
(1 + x)^n = 81
```

Big Union, Intersection, And etc

In fact, analogous to the Big sum and Big product notation (See Big Sum), most other operators may be denoted for aggregation by using the giant form of it with a dummy index.

For example: Assume A_1, A_2, A_3 are sets, then the Big Union can be used to show aggregation of several unions

$$\bigcup_{i=1}^{3} A_i = A_1 \cup A_2 \cup A_3$$

The Big Intersection can be used to show aggregation of several intersections

$$\bigcap_{i=1}^{3} A_i = A_1 \cap A_2 \cap A_3$$

If p_1, p_2, p_3 are Booleans, Big And shows aggregation of several Logical Ands

$$\bigwedge_{i=1}^{3} p_i = p_1 \wedge p_2 \wedge p_3$$

For more information on Logical And see the Logic notebook

Aggregating over collections

Until now all aggegration operations traverses a continious stretch of integers. Sometimes, traversal over other forms of indices is needed. In such cases, traversals over sets are indicated by showing membership of the index:

$$\sum_{i \in A} i^2$$

This can be expanded explicitely by defining set A, for example consider set $A = \{3, 6, 1.7, 5\}$, then

$$\sum_{i \in A} i^2 = 3^2 + 6^2 + 1.7^2 + 5^2$$

Note: This is true for all the above aggegation notations

[37]:
```
A = {3, 6, 1.7, 5}

# Σ_{a   A} a^2
total = sum(a ** 2 for a in A)
```

```
print('A:', A)
print('sum of a^2 for a in A:', total)
```

```
A: {1.7, 3, 5, 6}
sum of a^2 for a in A: 72.89
```

Einstein notations

In some contexts, especially in tensor algebra, it is convenient to write sums without explicitly writing the summation symbol $\sum$. In Einstein notation, an index that appears once up and once down is understood to be summed over.

For example, consider an $m \times m$ matrix A and vectors u and v of dimension m. Matrix-vector multiplication can be written as

$$u_i = A_{ij} v_j$$

which is shorthand for

$$u_i = \sum_{j=1}^{m} A_{ij} v_j$$

For the concrete case $m = 3$, this means:

$$u_1 = \sum_{j=1}^{3} A_{1j} v_j, \quad u_2 = \sum_{j=1}^{3} A_{2j} v_j, \quad u_3 = \sum_{j=1}^{3} A_{3j} v_j$$

For more information on tensors and Einstein notation, see the Linear Algebra notebook.

[38]:
```
# Einstein notation: u_i = A_ij * v_j (implied sum over j)
A = [[1, 2, 3],
     [4, 5, 6],
     [7, 8, 9]]

v = [2, 2, 2]

u = []
for i in range(len(A)):
    total = 0
    for j in range(len(v)):
        total += A[i][j] * v[j]
    u.append(total)
u
```

[38]:
```
[12, 30, 48]
```

Another common use of Einstein notation is the *trace* of a square matrix. If A is an $m \times m$ matrix, then the repeated index a_{ii} denotes a summation over the diagonal:

$$a_{ii} = \sum_{i=1}^{m} a_{ii} = a_{11} + a_{22} + \cdots + a_{mm}$$

This is the trace of A, often written $\mathrm{tr}(A)$.

[39]:
```
# Trace of A using Einstein notation: a_ii

A = [[1,2,3],
     [4,5,6],
     [7,8,9]]

trace = sum(A[i][i] for i in range(len(A)))
trace
```

[39]: 15